
THROUGH THE LENS MEDIA

Access Controls Policy

Lumière Ledger — Financial Management Platform

Document Version:	1.0
Effective Date:	May 18, 2026
Classification:	Confidential — Compliance Use
Owner:	Joshua Deuermeyer, Owner & Developer
Contact:	joshua.deuermeyer@gmail.com
Related Policy:	Information Security Policy v2.0

Confidential — restricted to authorized personnel and compliance reviewers.

1. Purpose and Scope

This Access Controls Policy defines how Through The Lens Media (TTLM) restricts and manages access to production systems, sensitive data, and the Lumière Ledger financial management platform. It applies to all infrastructure, databases, APIs, and third-party service integrations — including the pending Plaid bank connectivity integration.

The objective is to enforce least-privilege access, ensure all access is authenticated and authorized, and maintain audit trails for all privileged operations.

2. Documented Access Control Framework

Policy Statement

Access to all production assets — virtual infrastructure, databases, application APIs, and sensitive financial data — is granted on a need-to-know, least-privilege basis. All access requires authentication. No resource is accessible without explicit authorization. Access rights are reviewed on a quarterly basis.

Governing Principles

- Least Privilege — every component and user receives only the permissions required for its specific function
- Defense in Depth — multiple independent access control layers enforce the same boundaries
- Zero Trust — no request is trusted by default; every API call requires a valid JWT regardless of network origin

- Separation of Duties — client-facing credentials (Supabase anon key) cannot access privileged operations; server-side service role key is never exposed to clients
- Auditability — all authentication events and privileged operations are logged

3. Role-Based Access Control (RBAC)

Lumière Ledger implements RBAC at three enforcement layers: the database, the API middleware, and the frontend routing layer. Access decisions are made independently at each layer.

Role Definitions

Role	Description	Granted To	Enforcement
Standard User	Access to own data only — no cross-authentication	Authenticated users	Supabase RLS (user_id = auth.uid())
Administrator	Elevated API access — admin routes, subscription management, contact forms	Joshua Deuermeyer@gmail.com, requireRole() middleware + email check	
Service Role	Database bypass for server-side operations	Express API server (client-side only)	Server-side only; never returned to browser
Anonymous	Read-only access to public routes — unauthenticated requests	Unauthenticated requests	Supabase anon key + RLS deny-all on financial tables

RBAC Implementation Detail

- The user_roles table stores role assignments. The service role client (server-side only) reads this table to resolve roles — bypassing RLS only for this lookup
- requireRole('admin') middleware rejects requests with a 403 before any admin route handler executes
- All financial data tables (expenses, invoices, assets, leads, settings, receipts) have RLS policies that unconditionally filter by user_id = auth.uid() — a user cannot read, write, or delete another user's rows regardless of role
- The Express API adds a secondary .eq('user_id', req.user.id) filter on all destructive operations as defense-in-depth beyond RLS

4. Periodic Access Reviews and Audits

Review Schedule

Review Type	Frequency	Scope	Owner
Administrative Access Review	Quarterly	Verify Vercel, Supabase, GitHub, Resend admin accounts, remove stale sessions	Joshua Deuermeyer
RLS Policy Audit	Quarterly	Run rls-audit.sql in Supabase SQL Editor — verify all tables have RLS enabled and no policies are missing	Joshua Deuermeyer
API Key Rotation Review	Semi-annual	Review all active API keys (Supabase, Gemini, Resend, Blaid) and rotate any that are > 12 months old	Joshua Deuermeyer
Environment Variable Audit	On every deploy	Confirm no secrets are committed to source code via GitHub Actions (GitLab)	Automated (GitHub)
Dependency Access Audit	Bi-weekly	npm audit — identify packages with excessive permissions	Automated (CI/CD)

Audit findings are documented and remediated within the SLA defined in the Information Security Policy (critical: 7 days, medium: 30 days). All access reviews are performed by the owner and developer, Joshua Deuermeyer, who retains complete accountability.

5. Zero Trust Access Architecture

Lumière Ledger implements zero trust principles at every layer of the stack. No request — regardless of its origin, IP, or network — is trusted without explicit cryptographic verification.

Zero Trust Controls in Place

Control	Implementation	Layer
Identity verification on every request	JWT validated via Supabase JWKS on every API call — no session cookies	API Middleware
Per-row data isolation	RLS policy: <code>user_id = auth.uid()</code> enforced by PostgreSQL on every query	Database
Short-lived tokens	Supabase JWTs expire per configured TTL; <code>autoRefreshToken</code> renewal disabled	Auth Layer
No implicit network trust	Vercel serverless functions are stateless — no persistent connections that could be hijacked between requests	Infrastructure
Service key isolation	The Supabase service role key (which bypasses RLS) exists only as a secret environment variable — it is never exposed	Secret Management
HTTPS enforced	All traffic requires TLS 1.2+; HTTP requests are redirected. No plaintext transmission is possible	Transport

6. Centralized Identity and Access Management

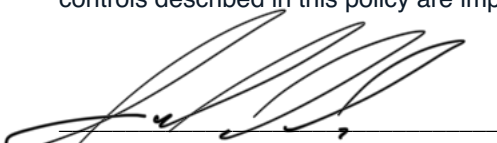
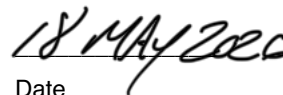
All identity and access management for Lumière Ledger is centralized through Supabase Auth, which serves as the single source of truth for user identities, session tokens, and authentication state.

Centralized IAM Capabilities

- Single sign-on (SSO) via Google OAuth 2.0 — no separate password required for Google users
- Email/password authentication with bcrypt-hashed credential storage — passwords are never stored or transmitted in plaintext
- JWT issuance and JWKS-based validation — all downstream systems (Express API) validate tokens against the same Supabase JWKS endpoint
- Session management: `autoRefreshToken` + `persistSession` — sessions survive app restarts without requiring re-authentication
- MFA support available via Supabase TOTP — to be enforced for all users prior to Plaid Link activation
- Centralized user management — account creation, deletion, and suspension all flow through Supabase Auth admin API
- Audit log: all authentication events (logins, failures, token refreshes) are retained in Supabase Auth logs

Policy Attestation

I, Joshua Deurmeyer, as the sole owner and operator of Through The Lens Media, attest that the access controls described in this policy are implemented and operational as of the effective date above.


Joshua Deurmeyer
Date

Owner & Developer, Through The Lens Media

Through The Lens Media • Access Controls Policy v1.0 • Effective May 18, 2026 • Confidential